

Project Report Format

1. INTRODUCTION

The HDI Predictor project estimates Human Development Index values from measurable country indicators using a Linear Regression model and a Flask web application.

The project was developed as an internship-style machine learning workflow that progresses from dataset understanding to training, evaluation, and browser-based presentation.

1.1 Project Overview

The project uses life expectancy, expected years of schooling, mean years of schooling, and gross national income per capita as input indicators.

A cleaned CSV dataset and a saved pickle model support prediction through the Flask interface.

1.2 Purpose

The purpose of the project is to provide a simple and understandable browser-based system for HDI estimation. It demonstrates how a regression model can be connected to a web application for interactive prediction.

2. IDEATION PHASE

The ideation phase focused on defining the HDI estimation problem, identifying the target users, and selecting ideas that could move data from raw CSV input to a usable web interface.

2.1 Problem Statement

The project needs to estimate a country's HDI from measurable indicators because raw development data is spread across multiple columns and years.

The solution should reduce manual comparison and provide an automated browser-based workflow.

2.2 Empathy Map Canvas

The user wants a quick and reliable HDI estimate, sees indicator values and charts, says and does input selection and CSV upload, and needs a clear explanation of the output.

Pain points include scattered data and notebook-only analysis, while gains include browser access, visual output, and faster interpretation.

2.3 Brainstorming

The brainstorming stage prioritized Linear Regression, Flask integration, country auto-fill, batch prediction, and chart-based explanation.

3. REQUIREMENT ANALYSIS

Requirement analysis translated the project idea into functional and non-functional requirements, data flow, planning, and stack selection.

3.1 Customer Journey map

The customer journey begins with opening the home page, continues with entering or selecting country indicators, and ends with receiving an HDI score, category, and visual result.

3.2 Solution Requirement

The solution requires data preparation, prediction input handling, model inference, visualization, batch prediction, and country auto-fill, along with usability, reliability, maintainability, security, and scalability.

3.3 Data Flow Diagram

The data flow starts from HDI.csv, moves through preprocessing and model training, stores the trained HDI.pkl file, and ends with Flask routes and browser rendering.

3.4 Technology Stack

The project uses Python, Flask, Pandas, NumPy, Scikit-Learn, Matplotlib, Seaborn, HTML, CSS, JavaScript, Git, GitHub, and pickle-based model storage.

4. PROJECT DESIGN

The design phase describes the fit between the problem and the solution, the proposed implementation, and the overall architecture.

4.1 Problem Solution Fit

The solution fits the problem because it transforms country indicators into a reusable prediction workflow and reduces manual comparison by combining preprocessing, modeling, and Flask presentation.

4.2 Proposed Solution

The proposed solution is a Flask-based application backed by a Linear Regression model that supports single predictions, batch CSV prediction, and country auto-fill.

4.3 Solution Architecture

The architecture connects the notebook, cleaned dataset, trained model, Flask routes, templates, and JavaScript charts into one modular system.

5. PROJECT PLANNING & SCHEDULING

The project was planned in sprints covering data preparation, EDA, feature engineering, model training, evaluation, Flask integration, frontend development, and testing.

5.1 Project Planning

Sprint planning divided the work into manageable tasks with story points so that the 15-day project could be tracked clearly from start to finish.

6. FUNCTIONAL AND PERFORMANCE TESTING

Testing validated numeric input handling, country auto-fill, single prediction, CSV batch prediction, response time, and model loading.

6.1 Performance Testing

The project was checked for quick response, stable model loading, and smooth processing of batch requests.

7. RESULTS

The completed application runs locally through Flask and produces HDI scores, categories, confidence values, and charts based on user input.

7.1 Output Screenshots

The interface shows the home page, prediction form, result page, charts, and batch upload workflow.

8. ADVANTAGES & DISADVANTAGES

Advantages include a simple browser interface, reusable model logic, visual explanation, and batch prediction support.

Limitations include local deployment, reliance on a pre-trained dataset, and the need for future improvements if new indicators are added.

9. CONCLUSION

The HDI Predictor successfully connects machine learning and web development to produce an understandable HDI estimation system.

The project demonstrates a complete path from data preparation to user-facing prediction.

10. FUTURE SCOPE

Future versions can add more datasets, stronger models, deployment support, richer analytics, and additional visualization features.

11. APPENDIX

Source code, dataset references, and project demo links can be added here.

Source Code(if any)

Available in the project repository and Flask application folders.

Dataset Link

HDI.csv and HDI_Cleaned.csv are used for preprocessing and prediction.

GitHub & Project Demo Link

The repository and local demo can be attached for submission.